Topic: Instruction Formats

Q. What is an Instruction Format? Explain the different types of instruction formats based on the number of addresses (Zero, One, Two, and Three-Address instructions) with examples.

> 📌 **Definition to Remember**
> An **Instruction Format** defines the binary layout of a machine instruction. It typically consists of an **Opcode** (the operation to perform, e.g., ADD) and **Operand(s)** (the data or memory addresses involved). Based on CPU architecture, instructions can have zero, one, two, or three address fields.

---

1. Zero-Address Instruction

Used in **Stack-based architectures**. The operands are implicitly located at the top of the stack.
* **Format:** `[ Opcode ]`
* **Example:** `ADD` (Pops the top two stack elements, adds them, and pushes result back).
* **Expression (Z = X + Y):** `PUSH X`, `PUSH Y`, `ADD`, `POP Z`.

2. One-Address Instruction

Used in **Accumulator-based architectures**. One operand is specified; the second operand and the destination are implicitly the Accumulator register.
* **Format:** `[ Opcode | Address 1 ]`
* **Example:** `ADD B` (Meaning: `ACC = ACC + M[B]`).
* **Expression (Z = X + Y):** `LOAD X`, `ADD Y`, `STORE Z`.

3. Two-Address Instruction

Used in **General Purpose Register architectures** (like Intel x86). It specifies two addresses, where one usually acts as both a source and the destination.
* **Format:** `[ Opcode | Destination | Source ]`
* **Example:** `ADD R1, R2` (Meaning: `R1 = R1 + R2`).

4. Three-Address Instruction

Used in modern **RISC architectures** (like ARM, MIPS). It specifies three distinct locations (one destination, two sources). Makes expressions short but requires longer instruction strings.
* **Format:** `[ Opcode | Destination | Source 1 | Source 2 ]`
* **Example:** `ADD R1, R2, R3` (Meaning: `R1 = R2 + R3`).

> ⭐ **Must-Write Points (for 10 marks)**
> 1. **Instruction Format** = Opcode (operation) + Operand (data/addresses).
> 2. Formats are classified by the number of addresses: 0, 1, 2, or 3.
> 3. **Zero-Address**: Stack-based; operands popped from stack (`ADD`).
> 4. **One-Address**: Accumulator-based; implicitly uses ACC (`ADD B` → ACC=ACC+B).
> 5. **Two-Address**: Register-based; one address is source+destination (`ADD R1, R2`).
> 6. **Three-Address**: RISC-based; specifies 2 sources and 1 destination (`ADD R1, R2, R3`).

7. Fewer addresses = simpler CPU but longer programs; more addresses = complex CPU but shorter programs.

Quick Recall

`Instruction = Opcode + Operand → Zero (Stack: ADD) → One (Accumulator: ADD B) → Two (x86: ADD R1, R2) → Three (RISC: ADD R1, R2, R3)`